

UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

DIPARTIMENTO DI INFORMATICA

CORSO DI LAUREA IN INFORMATICA

TESI DI LAUREA
IN
METODI AVANZATI DI PROGRAMMAZIONE

Sintesi di un algoritmo per Windows PE malware detection

RELATRICE:
Prof.ssa Annalisa Appice

LAUREANDO:
Domenico Bianchini

CORRELATRICE:
Prof.ssa Giuseppina Andresini

ANNO ACCADEMICO 2025 - 2026

Indice

1 Introduzione 1

1.1 Contesto e motivazioni 1

1.2 Obiettivi della tesi 2

1.3 Struttura della tesi 2

2 Malware e file Windows PE 4

2.1 Il formato Windows PE 4

2.2 Malware e tecniche di rilevamento 7

2.2.1 Tipologie di malware 7

2.2.2 Tecniche di rilevamento 8

3 Scoperta di Windows PE malware 10

3.1 Conversione di file PE in immagini 10

3.1.1 Rappresentazione in scala di grigi e RGB 10

3.1.2 Strategie di mapping 12

3.1.3 Dimensioni delle immagini 15

3.2 ResNet per classificazione di immagini 16

3.2.1 Reti neurali convoluzionali 16

3.2.2 ResNet e residual learning 18

4 Validazione empirica 21

4.1 Dataset 21

4.2 Set-up e metriche 22

4.2.1 Configurazioni sperimentali 22

4.2.2 Preparazione dei dati e addestramento 24

4.2.3 Valutazione e metriche 25

4.3 Risultati 27

5 Conclusioni e sviluppi futuri 28

Bibliografia 29

Capitolo 1

Introduzione

1.1 Contesto e motivazioni

La sicurezza informatica rappresenta un aspetto sempre più rilevante in un contesto in cui sistemi e servizi digitali sono utilizzati in numerosi ambiti della vita quotidiana. Tra le principali minacce alla sicurezza dei sistemi informatici rientrano i malware, software sviluppati con l'obiettivo di compiere azioni dannose o non autorizzate, che possono compromettere il corretto funzionamento dei sistemi e mettere a rischio i dati in essi contenuti. La diffusione dei malware e la loro continua evoluzione rendono il loro rilevamento un problema di particolare importanza nell'ambito della cybersecurity.

Esistono diversi approcci per il rilevamento dei malware, fra cui i metodi tradizionali basati sul riconoscimento di firme note e sull'analisi delle caratteristiche del software; tuttavia, la continua comparsa di nuove varianti di malware e l'utilizzo di tecniche che ne rendono più difficile l'individuazione hanno portato alla ricerca di approcci alternativi. In particolare, negli ultimi anni hanno assunto particolare importanza le tecniche basate su Machine Learning e Deep Learning, che permettono di individuare più efficacemente anche nuove varianti di malware, superando alcuni dei limiti dei metodi tradizionali.

L'utilizzo del Deep Learning consente, inoltre, di analizzare i dati attraverso diverse forme di rappresentazione, in particolare la rappresentazione visuale dei file malware, ottenuta trasformando le informazioni contenute nei file binari in immagini. In questo modo è possibile applicare tecniche e modelli di Deep Learning

per la classificazione delle immagini, con l'obiettivo di distinguere file malevoli da file legittimi.

1.2 Obiettivi della tesi

L'obiettivo di questo lavoro di tesi è la sintesi di un algoritmo in grado di classificare i file Windows Portable Executable (PE) come malevoli o benigni, rappresentandoli prima sotto forma di immagini e utilizzando successivamente un modello di Deep Learning per la loro classificazione.

In particolare, questo lavoro analizza e confronta tre diverse strategie per la conversione dei file PE in immagini, ossia LINEAR, ZIGZAG e SERPENTINE, che si differenziano in base al tipo di mapping utilizzato per costruire l'immagine. Per ciascuna strategia vengono confrontate sia immagini RGB sia grayscale, generate secondo due diverse configurazioni a dimensione fissa oppure variabile in base alla dimensione del file PE. Le immagini ottenute vengono poi utilizzate per addestrare e valutare un modello ResNet-50, con l'obiettivo di distinguere i file malevoli da quelli benigni.

Il confronto dei risultati ottenuti dalle diverse configurazioni e strategie di mapping permette infine di valutarne le prestazioni e individuare quale configurazione risulti più efficace nella classificazione dei file PE.

1.3 Struttura della tesi

La tesi è organizzata in cinque capitoli, il primo dedicato all'introduzione del lavoro, al suo contesto, alle motivazioni e agli obiettivi.

Il Capitolo 2 descrive il formato Windows Portable Executable (PE), analizzandone le principali caratteristiche, e introduce il concetto di malware e le principali tecniche utilizzate per il loro rilevamento.

Il Capitolo 3 descrive il processo di conversione dei file PE in immagini, presentando le diverse strategie di mapping e le configurazioni utilizzate per la loro rappresentazione, e l'utilizzo di ResNet per la successiva classificazione.

Il Capitolo 4 è dedicato alla validazione empirica del lavoro e descrive il dataset utilizzato, il set-up sperimentale e le metriche adottate per la valutazione. Vengono

inoltre analizzati e confrontati i risultati ottenuti dalle diverse configurazioni utilizzate, con lo scopo di individuare quella con le migliori prestazioni, ovvero quella più efficace nella classificazione dei file PE.

Infine, il Capitolo 5 riassume i principali risultati ottenuti e presenta le conclusioni del lavoro e i possibili sviluppi futuri.

Capitolo 2

Malware e file Windows PE

2.1 Il formato Windows PE

Il Portable Executable (PE) è il formato utilizzato nei sistemi Windows per i file `.exe`, `.dll` e altre tipologie di file eseguibili. Inizialmente progettato come formato eseguibile standard per i sistemi Windows, il formato PE, fin dalla sua introduzione, ha subito diverse modifiche, anche in seguito al passaggio ai sistemi Windows a 64 bit.

La struttura di un file PE contiene le informazioni necessarie al sistema operativo Windows per caricare e gestire il codice eseguibile. Oltre al codice da eseguire, un file PE contiene infatti metadati e altre informazioni organizzate secondo una struttura ben definita, la cui conoscenza riveste un ruolo importante nell'ambito dell'analisi dei malware.

In particolare, la struttura di un file PE è organizzata in tre parti principali [1], come mostrato nella Figura 2.1:

- **Header**: contiene le informazioni principali relative al file PE e al suo caricamento in memoria. L'header è a sua volta suddiviso nelle seguenti parti:
  - **DOS Header**: rappresenta la parte iniziale del file PE e contiene informazioni relative al formato DOS. Contiene inoltre un riferimento che indica la posizione iniziale del PE Header;
  - **DOS Stub**: viene utilizzato per visualizzare un messaggio che indica se il programma può essere eseguito o meno in modalità DOS;

  - **NT Header**: contiene le principali informazioni relative al file PE ed è a sua volta suddiviso in:
    * **PE Signature**: identifica il file come Portable Executable;
    * **COFF File Header**: contiene informazioni sulle caratteristiche del file e sulle sezioni presenti;
    * **Optional Header**: contiene informazioni utilizzate durante il caricamento del file in memoria, tra cui il punto di ingresso del programma.

- **Tabella delle sezioni**: descrive le caratteristiche delle diverse sezioni presenti nel file e contiene le informazioni necessarie per mapparle in memoria e indicare se ciascuna sezione è di sola lettura oppure scrivibile;

- **Sezioni**: contengono il codice e i dati del file PE. Tra le sezioni comunemente presenti si trovano:
  - `.text`: contiene il codice eseguibile;
  - `.data`: contiene le variabili globali e statiche inizializzate;
  - `.bss`: contiene le variabili globali e statiche non inizializzate;
  - `.rsrc`: contiene le risorse utilizzate dal programma, come icone, immagini, stringhe e finestre di dialogo;
  - `.reloc`: contiene i dati necessari alla rilocazione del programma;

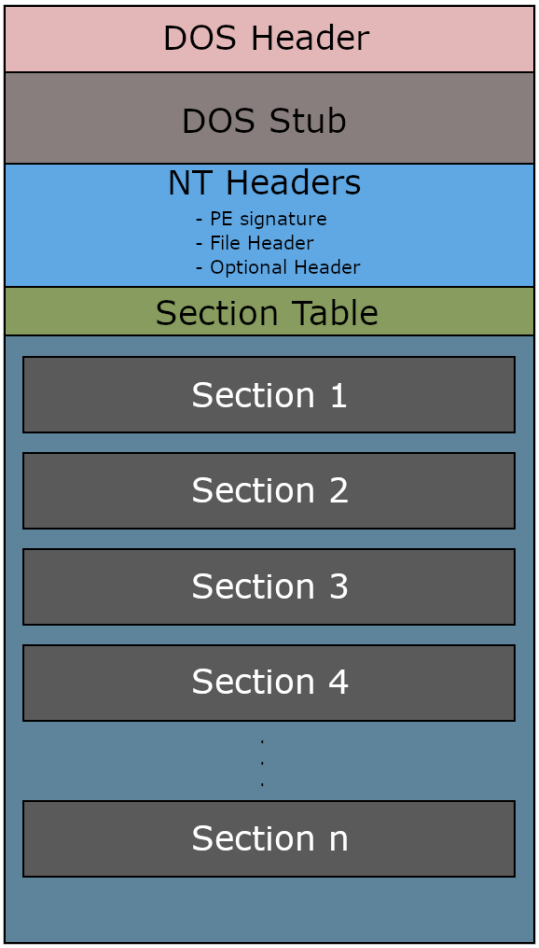


Figura 2.1: Struttura generale di un file PE, tratta da [2]

L'header riveste un ruolo importante nella classificazione dei malware, dato che le informazioni in esso contenute sono risultate tra quelle che contribuiscono maggiormente al processo di classificazione [3].

L'analisi della struttura delle sezioni può essere utile anche nell'ambito del rilevamento dei malware, dato che il codice malevolo può essere nascosto all'interno di sezioni insolite oppure possono essere aggiunte ulteriori sezioni con l'obiettivo di rendere più complessa l'analisi del file. Inoltre, i file malware possono presentare anomalie nella struttura e nelle caratteristiche del formato PE rispetto ai file benigni, rendendo queste informazioni rilevanti per il loro rilevamento [4].

2.2 Malware e tecniche di rilevamento

Il termine malware deriva da *malicious software* e indica un software progettato con lo scopo di compromettere la riservatezza, l'integrità o la disponibilità dei dati, delle applicazioni o del sistema operativo [5].

2.2.1 Tipologie di malware

I malware possono essere suddivisi in diverse tipologie in base alle loro caratteristiche e al loro comportamento. Inoltre, queste categorie non sono esclusive, poiché uno stesso malware può presentare contemporaneamente caratteristiche appartenenti a più tipologie [6].

Tra le principali tipologie di malware abbiamo i *worm*, che sono programmi in grado di essere eseguiti in modo indipendente e con la capacità di propagarsi verso altri sistemi. I worm possono inoltre essere utilizzati per creare *botnet*, impiegate per l'invio di messaggi di spam o per effettuare attacchi di tipo Distributed Denial of Service (DDoS), nei quali più computer infettati da worm agiscono in modo coordinato per esaurire le risorse di un sistema o la sua banda di rete.

I *virus*, a differenza dei worm, non possono essere eseguiti in modo indipendente, ma inseriscono il proprio codice malevolo all'interno di altri programmi e si attivano quando viene eseguito il programma infettato. Possono inoltre propagarsi infettando altri file presenti nel sistema e, attraverso file presenti su server condivisi, diffondersi anche verso altri computer.

I *Trojan horse* sono programmi che si presentano come software legittimi, ma che eseguono operazioni malevole in background. Una volta installati, possono ad esempio scaricare altri malware, modificare le impostazioni del sistema o infettare altri file.

Gli *spyware* sono malware utilizzati per raccogliere informazioni sensibili dal sistema infetto e trasmetterle all'attaccante. Le informazioni raccolte possono comprendere credenziali di accesso, dati bancari, pagine web visitate oppure il contenuto di documenti ed e-mail.

I *bot* permettono a un attaccante, chiamato *bot master*, di controllare da remoto il sistema infetto. Un insieme di bot controllati dallo stesso *bot master* costituisce

una *botnet*, che può essere utilizzata, ad esempio, per l'invio di messaggi di spam o per attività di spyware.

Infine, i *rootkit*, la cui principale caratteristica è quella di nascondere determinate informazioni all'utente del sistema, come processi, file o connessioni di rete. Per questo motivo, molti malware utilizzano tecniche proprie dei rootkit per nascondere la propria presenza all'interno del sistema.

2.2.2 Tecniche di rilevamento

Il rilevamento dei malware può essere effettuato attraverso diverse tecniche. Uno degli approcci tradizionalmente utilizzati, anche dagli antivirus, è quello basato sulle firme, in cui i file vengono confrontati con un insieme di firme associate a malware già conosciuti.

Questo approccio presenta tuttavia alcuni limiti, in particolare le firme vengono generalmente create da analisti, richiedendo tempo e potendo essere soggette a errori. Inoltre, i metodi basati sulle firme non sono in grado di rilevare minacce sconosciute per le quali non è ancora disponibile una firma, dato che solo quando viene individuata una nuova minaccia questa può essere analizzata e può essere creata una nuova firma, che viene successivamente aggiunta al database utilizzato per il rilevamento [6].

Per affrontare alcuni dei limiti dei metodi basati sulle firme, sono state sviluppate altre tecniche per il rilevamento dei malware. In particolare, è possibile distinguere due principali approcci: l'analisi dinamica e l'analisi statica, in base all'esecuzione o meno del programma durante la raccolta delle feature, ovvero delle caratteristiche utilizzate per effettuare l'analisi [3].

L'analisi dinamica prevede l'esecuzione del programma e l'osservazione del suo comportamento durante l'esecuzione. In questo modo possono essere raccolte diverse informazioni, tra cui le sequenze delle chiamate API, le comunicazioni di rete, lo stato della memoria e della CPU. Questo tipo di analisi può inoltre contribuire al rilevamento di malware precedentemente sconosciuti e risulta meno vulnerabile alle tecniche di offuscamento del codice.

Tuttavia, presenta alcuni limiti, dato che ogni campione deve essere eseguito per un determinato periodo di tempo e può essere analizzato un singolo percorso di esecuzione del programma [4].

A differenza dell'analisi dinamica, l'analisi statica permette di analizzare un programma senza eseguirlo e, per questo motivo, richiede tempi inferiori per l'analisi. Dato che generalmente il codice sorgente dei malware non è disponibile, le tecniche di analisi statica applicabili devono ricavare le informazioni direttamente dalla rappresentazione binaria del programma.

Anche l'analisi statica presenta alcuni limiti, dato che lavorare direttamente sulla rappresentazione binaria di un programma può risultare complesso. Inoltre, i malware possono utilizzare tecniche in grado di modificare il programma e ostacolare questo tipo di analisi [6].

Tabella 2.1: Confronto tra analisi statica e analisi dinamica dei malware

Caratteristica	**Analisi statica**	**Analisi dinamica**
Esecuzione	Non richiesta	Richiesta
Informazioni analizzate	Rappresentazione binaria del programma	Comportamento durante l'esecuzione
Esempi di feature	Informazioni ricavate dal contenuto del file	Chiamate API, comunicazioni di rete, memoria e CPU
Vantaggi	Tempi di analisi inferiori	Può contribuire al rilevamento di malware sconosciuti ed è meno vulnerabile alle tecniche di offuscamento
Limiti	Può essere ostacolata da tecniche di packing o cifratura	Può richiedere tempi di analisi maggiori

I limiti degli approcci descritti in precedenza, insieme alla crescente complessità e quantità dei malware, hanno favorito l'utilizzo di tecniche basate sul Machine Learning. In particolare, queste tecniche permettono di apprendere modelli e pattern a partire dalle feature estratte dai malware, con l'obiettivo di supportarne il rilevamento e l'analisi [7].

Capitolo 3

Scoperta di Windows PE malware

3.1 Conversione di file PE in immagini

Un possibile approccio per l'analisi statica dei malware consiste nel rappresentare il contenuto binario dei file sotto forma di immagini. In questo modo, la sequenza di byte che compone un file può essere convertita in una rappresentazione visiva, sulla quale è possibile applicare tecniche di analisi e classificazione delle immagini.

3.1.1 Rappresentazione in scala di grigi e RGB

Nataraj et al. [8] hanno proposto un approccio in cui il contenuto binario di un file malware viene riorganizzato in una matrice e rappresentato sotto forma di un'immagine in scala di grigi. Gli autori hanno osservato che malware appartenenti alla stessa famiglia possono presentare immagini simili a livello di layout e texture. A partire da questa osservazione, hanno quindi utilizzato caratteristiche estratte dalle immagini per effettuare la classificazione, senza richiedere il disassemblaggio del codice o l'esecuzione dei file analizzati.

In particolare, la conversione in immagine del file PE viene eseguita leggendo il contenuto del file come una sequenza di byte, ciascuno dei quali viene rappresentato attraverso un valore intero compreso tra 0 e 255. Nel caso di un'immagine in scala di grigi, ogni byte viene associato a un pixel, il cui valore determina il livello di intensità, dove 0 corrisponde al nero e 255 al bianco. I valori ottenuti vengono quindi organizzati in una matrice bidimensionale che costituisce l'immagine.

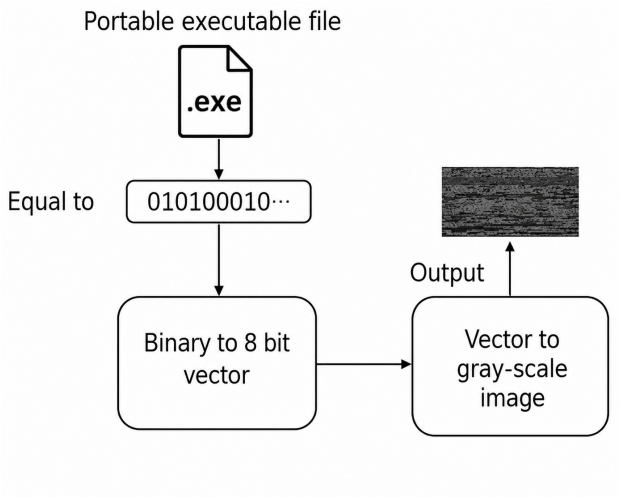
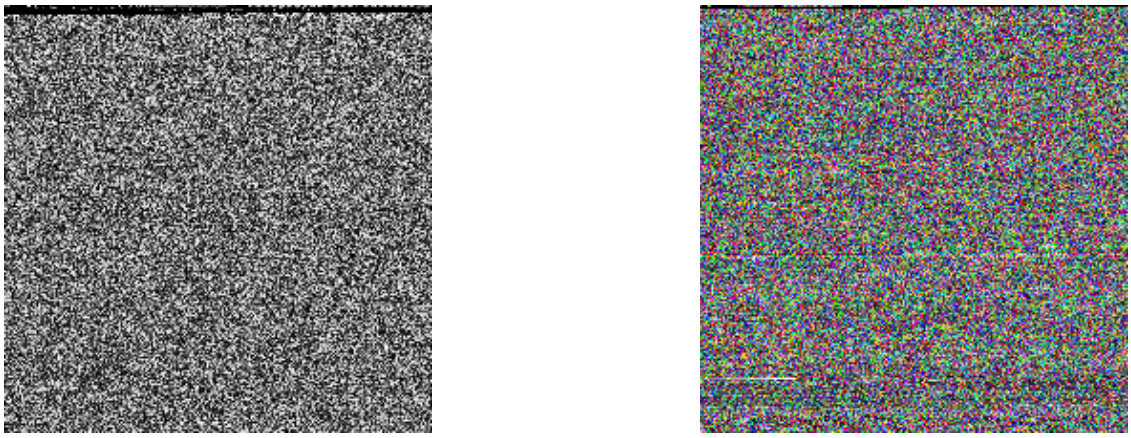


Figura 3.1: Principali fasi della conversione di un file PE in un'immagine in scala di grigi, tratta da [3]

Oltre alla rappresentazione in scala di grigi, i byte del file PE possono essere utilizzati per costruire un'immagine RGB. In questo caso, ciascun pixel è composto da tre canali, rosso, verde e blu, ognuno dei quali assume, come nel caso precedente, un valore compreso tra 0 e 255.

Guo et al. [9] utilizzano una rappresentazione RGB in cui tre byte consecutivi vengono raggruppati e associati, nello stesso ordine, ai tre canali di un pixel. Il primo byte determina quindi il valore del canale rosso, il secondo quello del canale verde e il terzo quello del canale blu.



(a) Rappresentazione in scala di grigi (b) Rappresentazione RGB

Figura 3.2: Rappresentazioni in scala di grigi e RGB dello stesso file PE

Nel presente lavoro di tesi vengono considerate entrambe le rappresentazioni descritte, in scala di grigi e RGB. Una volta ottenuti i valori dei pixel a partire dai byte del file PE, occorre stabilire come disporli all'interno della matrice che costituisce l'immagine.

3.1.2 Strategie di mapping

Linh et al. [10] analizzano differenti modalità di disposizione dei pixel, considerando tre diverse strategie di mapping denominate Normal, Zigzag e Serpentine.

Il mapping Normal, indicato nel presente lavoro come **LINEAR**, dispone i pixel seguendo l'ordine originale dei byte del file PE, procedendo da sinistra verso destra e dall'alto verso il basso. Questa disposizione, tuttavia, interrompe la continuità tra righe consecutive, poiché l'ultimo pixel di una riga e il primo della successiva, pur corrispondendo a byte consecutivi, non risultano adiacenti nell'immagine.

Algoritmo 3.1 Pseudocodice del mapping LINEAR

```
1: image = matrice di zeri di dimensione height × width × channels
2: index = 0
3: for y = 0 to height − 1 do
4:     for x = 0 to width − 1 do
5:         for c = 0 to channels − 1 do
6:             if index < length(byteData) then
7:                 image[y][x][c] = byteData[index]
8:                 index = index + 1
```

Il mapping **ZIGZAG** dispone invece i pixel alternando la direzione di percorrenza delle righe. Una riga viene quindi percorsa da sinistra verso destra, mentre quella successiva da destra verso sinistra. In questo modo viene mantenuta la continuità tra righe consecutive, poiché i pixel corrispondenti a byte consecutivi rimangono adiacenti nell'immagine.

Algoritmo 3.2 Pseudocodice del mapping ZIGZAG

```
1: image = matrice di zeri di dimensione height × width × channels
2: index = 0
3: for y = 0 to height − 1 do
4:     if y mod 2 = 0 then
5:         xRange = 0 to width − 1
6:     else
7:         xRange = width − 1 to 0
8:     for x in xRange do
9:         for c = 0 to channels − 1 do
10:            if index < length(byteData) then
11:                image[y][x][c] = byteData[index]
12:                index = index + 1
```

Il mapping **SERPENTINE** utilizza infine un percorso basato sulle diagonali della matrice, alternandone la direzione di percorrenza. Questa disposizione consente di mantenere adiacenti nell'immagine i pixel corrispondenti a byte consecutivi, distribuendoli però lungo le diagonali anziché lungo le righe.

Algoritmo 3.3 Pseudocodice del mapping SERPENTINE

```
1: image = matrice di zeri di dimensione height × width × channels
2: index = 0
3: for diagonal = 0 to height + width − 2 do
4:     startY = max(0, diagonal − (width − 1))
5:     endY = min(height − 1, diagonal)
6:     if diagonal mod 2 = 0 then
7:         yRange = startY to endY
8:     else
9:         yRange = endY to startY
10:    for y in yRange do
11:        x = diagonal − y
12:        for c = 0 to channels − 1 do
13:            if index < length(byteData) then
14:                image[y][x][c] = byteData[index]
15:                index = index + 1
```

Nel caso dei file PE, questa disposizione permette inoltre di concentrare in una stessa area dell'immagine le informazioni appartenenti alla parte iniziale del file, in cui è contenuto l'header, che comprende informazioni significative per la

classificazione dei file PE, invece di distribuirle lungo le prime righe dell'immagine come avviene nei mapping precedenti.

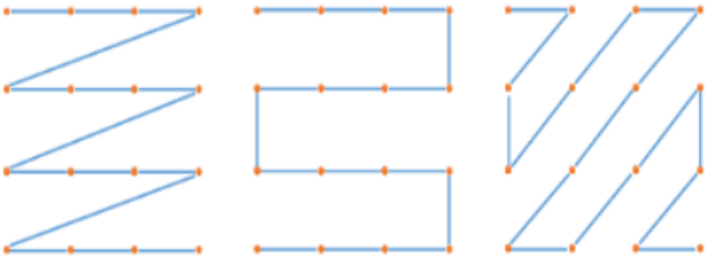


Figura 3.3: Disposizione dei pixel secondo i mapping Normal, Zigzag e Serpentine, tratta da [10]



Figura 3.4: Esempi di immagini malware ottenute utilizzando rispettivamente i mapping Normal, Zigzag e Serpentine, tratte da [10]

Tabella 3.1: Confronto tra le strategie di mapping considerate

Mapping	**Disposizione**	**Effetto della disposizione**
LINEAR	Da sinistra verso destra, riga per riga	Interrompe la continuità tra righe consecutive
ZIGZAG	Direzione alternata tra righe consecutive	Mantiene adiacenti i pixel corrispondenti a byte consecutivi
SERPENTINE	Percorso alternato lungo le diagonali	Mantiene l'adiacenza tra byte consecutivi e concentra la parte iniziale del file in una stessa area dell'immagine

3.1.3 Dimensioni delle immagini

Un ulteriore aspetto da considerare nella conversione riguarda la scelta delle dimensioni dell'immagine, dato che i file PE possono avere dimensioni differenti e, di conseguenza, anche il numero di pixel ottenuti a partire dalla sequenza di byte può variare da un file all'altro.

Una possibile soluzione consiste nel fissare a priori le dimensioni dell'immagine, stabilendo quindi un numero massimo di pixel disponibili. Se i dati ottenuti dal file non sono sufficienti a riempire l'intera immagine, è possibile applicare lo zero-padding, completando i pixel rimanenti con valori pari a zero. Al contrario, se la quantità di dati supera la capacità dell'immagine, i byte che non possono essere rappresentati vengono eliminati mediante troncamento [11].

Un approccio alternativo consiste nell'adattare le dimensioni dell'immagine alla quantità di dati da rappresentare, in modo da preservare l'intera sequenza di byte ed evitare il troncamento. Nataraj et al. [8], in particolare, scelgono la larghezza dell'immagine in base alla dimensione del file, utilizzando diversi intervalli prestabiliti, mentre l'altezza dipende dal rapporto tra il numero di byte da rappresentare e la larghezza scelta.

File Size Range	Image Width
<10 kB	32
10 kB – 30 kB	64
30 kB – 60 kB	128
60 kB – 100 kB	256
100 kB – 200 kB	384
200 kB – 500 kB	512
500 kB – 1000 kB	768
>1000 kB	1024

Figura 3.5: Larghezze delle immagini utilizzate in funzione della dimensione dei file, tratta da [8]

Nel presente lavoro vengono utilizzate entrambe le configurazioni per le dimensioni delle immagini, a dimensione fissa oppure con altezza variabile. Nella prima configurazione le immagini hanno dimensioni fisse pari a 224×224 pixel. Nella seconda, invece, la larghezza viene determinata a partire dalla lunghezza media dei

file del dataset e dal numero di canali della rappresentazione. La larghezza è stata calcolata tramite la seguente formula:

$$\text{larghezza} = \left\lceil \sqrt{\frac{\text{lunghezza media dei file}}{\text{numero di canali}}} \right\rceil .$$

Dividendo la lunghezza media dei file per il numero di canali si ottiene il numero medio di pixel necessari, considerando che a ciascun pixel corrisponde un byte nella rappresentazione in scala di grigi e tre byte nella rappresentazione RGB. La radice quadrata di tale valore viene quindi utilizzata per determinare la larghezza dell'immagine.

Una volta fissata la larghezza, per ciascun file viene prima calcolato il numero di pixel necessari in base al numero di byte del file e al numero di canali:

$$\text{numero di pixel} = \left\lceil \frac{\text{numero di byte del file}}{\text{numero di canali}} \right\rceil .$$

Nel caso RGB, se il numero di byte del file non è multiplo di tre, l'arrotondamento per eccesso permette di considerare anche gli eventuali byte rimanenti.

L'altezza viene quindi determinata dividendo il numero di pixel necessari per la larghezza e arrotondando il risultato per eccesso:

$$\text{altezza} = \left\lceil \frac{\text{numero di pixel necessari}}{\text{larghezza}} \right\rceil .$$

3.2 ResNet per classificazione di immagini

Una volta rappresentati i file PE sotto forma di immagini, il problema della distinzione tra file benigni e malevoli può essere affrontato come un problema di classificazione di immagini. A questo scopo possono essere utilizzate le reti neurali convoluzionali (Convolutional Neural Networks, CNN), modelli particolarmente adatti all'elaborazione di dati organizzati sotto forma di immagini.

3.2.1 Reti neurali convoluzionali

Le CNN sono costituite da una successione di livelli che elaborano progressivamente i dati ricevuti in input. Un ruolo centrale è svolto dai livelli convoluzionali, nei quali

dei filtri analizzano porzioni dell'immagine, andando a produrre delle feature map che evidenziano determinate caratteristiche presenti nell'immagine. Lo stesso filtro viene utilizzato nelle diverse parti dell'immagine, permettendo di individuare una determinata caratteristica indipendentemente dalla sua posizione [12].

Attraverso la successione di più livelli, la rete è in grado di individuare caratteristiche progressivamente più complesse dell'immagine. I primi livelli possono individuare caratteristiche semplici, come colori e contorni, mentre i livelli successivi combinano tali informazioni per riconoscere forme e strutture più complesse.

Il risultato prodotto dai livelli convoluzionali viene generalmente elaborato mediante l'utilizzo di una funzione di attivazione non lineare, come la Rectified Linear Unit (ReLU). Questa funzione assegna un valore pari a zero agli input negativi e mantiene invariati quelli positivi, introducendo non linearità nella rete e permettendo di apprendere pattern complessi e velocizzare l'addestramento.

Ai livelli convoluzionali possono inoltre essere alternati livelli di pooling, utilizzati per ridurre la dimensionalità delle feature map. L'operazione di pooling considera delle regioni dell'input e ne aggrega i valori, producendo una rappresentazione di dimensioni inferiori e permettendo quindi di ridurre la complessità computazionale della rete. Tra le principali operazioni di pooling vi sono il max pooling, che seleziona il valore massimo presente in ciascuna regione considerata, e l'average pooling, che ne calcola invece il valore medio [12].

Nella parte finale di una CNN possono essere presenti uno o più livelli completamente connessi (*fully connected*), nei quali ogni neurone è collegato a tutti i neuroni del livello precedente. Questi livelli utilizzano le caratteristiche estratte dai livelli precedenti per produrre l'output utilizzato per la classificazione.

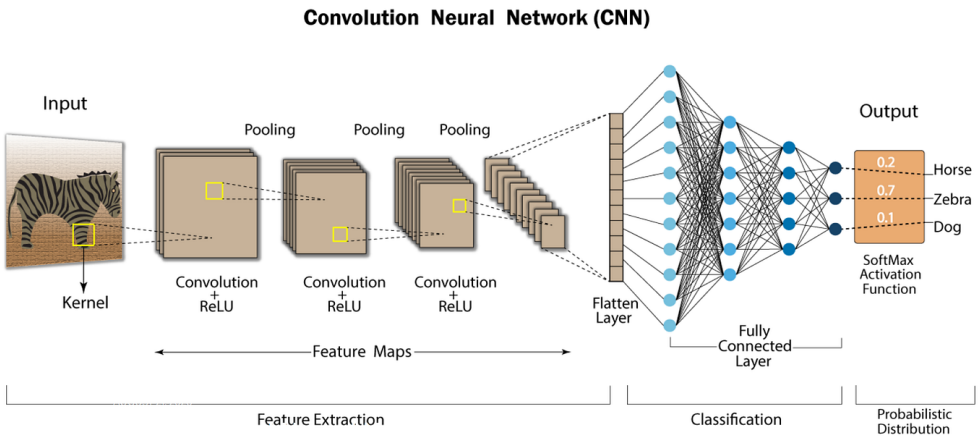


Figura 3.6: Esempio della struttura di una rete neurale convoluzionale, tratta da [13]

3.2.2 ResNet e residual learning

Nel corso degli anni sono state sviluppate diverse architetture di reti neurali convoluzionali, caratterizzate da strutture e profondità differenti. Tra queste rientra la Residual Network (ResNet), proposta da He et al. [14], che consente l'addestramento di reti estremamente profonde, cioè caratterizzate da un elevato numero di livelli.

L'aumento della profondità di una rete neurale non comporta tuttavia necessariamente un miglioramento delle prestazioni. Con l'aumentare del numero di livelli può infatti verificarsi il cosiddetto *degradation problem*, per cui l'accuratezza della rete smette inizialmente di migliorare e successivamente tende a diminuire. Come osservato dagli autori, questo fenomeno non è necessariamente dovuto all'overfitting, poiché l'aggiunta di ulteriori livelli può portare anche a un peggioramento delle prestazioni durante l'addestramento.

Per affrontare questo problema, ResNet introduce il concetto di *residual learning*. In una rete tradizionale, una successione di livelli apprende direttamente una mappatura $H(x)$ a partire dall'input x. ResNet modifica questo approccio facendo apprendere ai livelli una funzione residua $F(x)$, che rappresenta la differenza tra la mappatura desiderata e l'input:

$$F(x) = H(x) - x. \tag{3.1}$$

La mappatura originale può quindi essere espressa come

$$H(x) = F(x) + x. \tag{3.2}$$

In questo modo, invece di apprendere direttamente la mappatura $H(x)$, la successione di livelli apprende la funzione residua $F(x)$ rispetto all'input.

Questa formulazione viene realizzata attraverso le *shortcut connections*, collegamenti che saltano uno o più livelli della rete. Nel caso più semplice, la shortcut lascia invariato l'input x e lo aggiunge direttamente all'output della funzione residua $F(x)$. Queste connessioni non introducono parametri aggiuntivi né aumentano la complessità computazionale della rete.

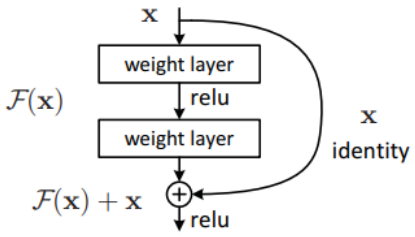


Figura 3.7: Schema di un blocco residuo con shortcut connection, tratto da [14]

Tra le diverse architetture basate sul residual learning, nel presente lavoro viene utilizzata ResNet-50, una rete composta da 50 livelli. Questa architettura utilizza un design a collo di bottiglia (*bottleneck design*), nel quale ciascun blocco residuo è composto da tre livelli convoluzionali con filtri rispettivamente di dimensione 1×1, 3×3 e 1×1 [14].

I livelli convoluzionali con filtri 1×1, posti rispettivamente prima e dopo quello con filtri 3×3, permettono di ridurre e successivamente ripristinare la dimensionalità dei canali, riducendo così la complessità computazionale del blocco.

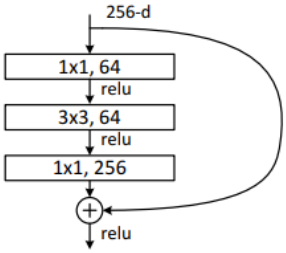


Figura 3.8: Schema del blocco bottleneck utilizzato in ResNet-50, tratto da [14]

La struttura complessiva di ResNet-50 è organizzata in più gruppi di blocchi residuali. Dopo i livelli iniziali di convoluzione e pooling, la rete presenta quattro gruppi di blocchi bottleneck, ripetuti rispettivamente 3, 4, 6 e 3 volte. La parte finale della rete comprende un livello di average pooling seguito da un livello fully connected utilizzato per la classificazione [14].

layer name	output size	18-layer	34-layer	50-layer	101-layer
conv1	112×112	7×7, 64, stride 2			
conv2_x	56×56	3×3 max pool, stride 2			
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax			

Figura 3.9: Confronto dell'architettura ResNet-50 con altre architetture ResNet, tratto da [14]

Capitolo 4

Validazione empirica

4.1 Dataset

Per la validazione sperimentale è stato utilizzato il *PE Malware Machine Learning Dataset* di Practical Security Analytics [15]. Il dataset comprende, oltre ai file PE, un file CSV contenente diverse informazioni associate a ciascun file, tra cui la lunghezza in byte e l'etichetta che indica l'appartenenza alla *Whitelist* o alla *Blacklist*, permettendo di distinguere rispettivamente i file benigni da quelli malevoli. Queste caratteristiche rendono il dataset particolarmente adatto al lavoro svolto, poiché mette a disposizione sia i file PE originali, necessari per la loro conversione in immagini, sia le relative etichette utilizzate per la classificazione.

Il dataset è composto complessivamente da 201.549 file PE, di cui 86.812 appartenenti alla *Whitelist* e quindi considerati benigni, e 114.737 appartenenti alla *Blacklist* e quindi considerati malevoli.

Ai fini della validazione sperimentale, il dataset è stato suddiviso in un training set, contenente l'80% dei campioni, e un test set, contenente il restante 20%. La suddivisione è stata effettuata in modo stratificato rispetto alla classe, così da mantenere la proporzione tra file benigni e malevoli presente nel dataset originale.

Tabella 4.1: Distribuzione dei campioni nei set di training e test

Insieme	**Benigni**	**Malevoli**	**Totale**
Training	69.450	91.789	161.239
Test	17.362	22.948	40.310
Totale	86.812	114.737	201.549

Il training set è stato utilizzato per l'addestramento del modello, mentre il test set è stato utilizzato per valutarne le prestazioni, dato che contiene dati non utilizzati durante il training e permette quindi di verificare le prestazioni del modello su nuovi campioni.

4.2 Set-up e metriche

4.2.1 Configurazioni sperimentali

A partire dai file PE presenti nel training set e nel test set sono state generate le immagini utilizzate per la validazione sperimentale. Sono state considerate tutte le combinazioni tra le tre strategie di mapping, **LINEAR**, **ZIGZAG** e **SERPENTINE**, le rappresentazioni RGB e in scala di grigi e le due configurazioni relative alle dimensioni delle immagini. Nella prima configurazione le immagini hanno dimensione fissa pari a 224×224 pixel, mentre nella seconda hanno larghezza fissa e altezza variabile in funzione della dimensione del file PE. Sono state quindi valutate complessivamente 12 configurazioni sperimentali.

Tabella 4.2: Configurazioni considerate nella validazione sperimentale

Mapping	**Rappresentazione**	**Larghezza**	**Altezza**
LINEAR	RGB	224	224
ZIGZAG	RGB	224	224
SERPENTINE	RGB	224	224
LINEAR	GRAY	224	224
ZIGZAG	GRAY	224	224
SERPENTINE	GRAY	224	224
LINEAR	RGB	457	Variabile
ZIGZAG	RGB	457	Variabile
SERPENTINE	RGB	457	Variabile
LINEAR	GRAY	792	Variabile
ZIGZAG	GRAY	792	Variabile
SERPENTINE	GRAY	792	Variabile

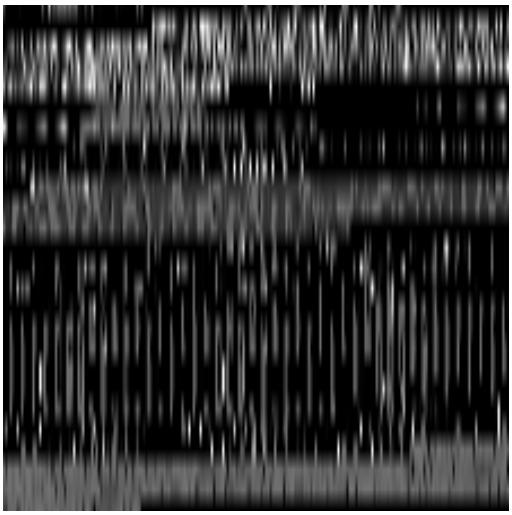
Nel caso delle immagini a dimensione fissa, i file PE sono stati convertiti direttamente in immagini di 224×224 pixel, che è la dimensione utilizzata in input dal modello ResNet-50 adottato negli esperimenti. Per questo non è stato necessario effettuare un successivo ridimensionamento.

Per le immagini ad altezza variabile, invece, la larghezza è stata determinata a partire dalla lunghezza media dei 201.549 file PE del dataset, che è risultata pari a 626.445,12 byte. Da questo valore sono state ottenute larghezze pari a 457 pixel per la rappresentazione RGB e a 792 pixel per quella in scala di grigi, secondo il procedimento descritto nel Capitolo 3, mentre l'altezza varia in base alla dimensione del singolo file PE.

Le immagini ad altezza variabile sono state successivamente ridimensionate a 224×224 pixel prima di essere fornite al modello. Il ridimensionamento è stato effettuato mediante interpolazione bilineare, utilizzando `nn.Upsample` di PyTorch [16].



(a) Dimensione fissa 224×224



(b) Altezza variabile dopo il resize a 224×224

Figura 4.1: Confronto dello stesso file PE con mapping LINEAR in scala di grigi nelle configurazioni a dimensione fissa e ad altezza variabile

Come si può osservare dalla Figura 4.1, nella configurazione a dimensione fissa è visibile la parte dell'immagine generata mediante zero-padding, mentre nella configurazione ad altezza variabile l'immagine appare maggiormente allungata in altezza in seguito al resize a 224×224 pixel.

4.2.2 Preparazione dei dati e addestramento

Durante la conversione dei file PE è stato inoltre generato, per il training set e per il test set, un file CSV contenente il percorso di ciascuna immagine e la relativa etichetta. In particolare, l'etichetta 0 identifica i file benigni, mentre l'etichetta 1 identifica i file malevoli. Questi file CSV sono stati successivamente utilizzati per associare ogni immagine alla classe corrispondente durante il caricamento dei dati.

Per il caricamento e la preparazione delle immagini sono stati utilizzati `Dataset` e `DataLoader` di PyTorch [16]. Il dataset associa ciascuna immagine alla relativa etichetta utilizzando le informazioni contenute nei file CSV, mentre il DataLoader organizza i campioni nei batch da fornire al modello durante il training e il test.

Durante il caricamento, tutte le immagini sono state lette in formato RGB. Nel caso delle immagini originariamente in scala di grigi, i valori dei pixel sono stati

quindi replicati sui tre canali, in modo da ottenere un'immagine compatibile con i tre canali di input utilizzati da ResNet-50.

Prima di essere fornite al modello, le immagini sono state convertite in valori compresi tra 0 e 1 e successivamente normalizzate utilizzando, per i tre canali, i valori medi $[0.485, 0.456, 0.406]$ e le deviazioni standard $[0.229, 0.224, 0.225]$, utilizzati dal modello `microsoft/resnet-50` [17].

Il training set descritto nella Sezione 4.1 è stato ulteriormente suddiviso, utilizzando l'80% dei campioni per l'addestramento e il restante 20% per la validazione. Il validation set è stato utilizzato per valutare le prestazioni del modello durante le diverse epoche di addestramento.

Per tutte le configurazioni sperimentali è stato utilizzato, tramite la libreria Hugging Face Transformers, il modello `microsoft/resnet-50`, pre-addestrato sul dataset ImageNet-1k [17]. Il modello utilizza l'architettura ResNet-50 descritta nel Capitolo 3 ed è stato adattato al lavoro svolto impostando il numero di classi di output a due, corrispondenti ai file benigni e malevoli.

L'addestramento è stato eseguito per 4 epoche utilizzando l'ottimizzatore Adam, con un learning rate pari a 0.001 e un batch size di 32 immagini, mentre come funzione di loss è stata utilizzata la Cross-Entropy Loss, calcolata sia durante il training sia sul validation set al termine di ogni epoca. Se la validation loss risultava inferiore al valore migliore registrato nelle epoche precedenti, il modello veniva salvato sostituendo quello precedente, in modo da mantenere al termine dell'addestramento il modello con la validation loss più bassa.

Gli esperimenti sono stati eseguiti nell'ambiente Kaggle utilizzando le GPU messe a disposizione dalla piattaforma per l'addestramento e il test del modello.

4.2.3 Valutazione e metriche

Il modello salvato al termine dell'addestramento è stato quindi utilizzato sul test set per ottenere le predizioni finali e valutarne le prestazioni. Confrontando le etichette predette dal modello con quelle reali è stata costruita la matrice di confusione, da cui sono stati ricavati i valori dei veri positivi (*true positive*, TP), dei veri negativi (*true negative*, TN), dei falsi positivi (*false positive*, FP) e dei falsi negativi (*false negative*, FN).

Considerando i file malevoli come classe positiva e quelli benigni come classe negativa, i veri positivi (TP) corrispondono ai file malevoli classificati correttamente e i veri negativi (TN) a quelli benigni classificati correttamente. I falsi positivi (FP) indicano invece i file benigni classificati come malevoli, mentre i falsi negativi (FN) indicano quelli malevoli classificati come benigni.

A partire dai valori della matrice di confusione sono state calcolate precision, recall e F1-score per entrambe le classi.

Per la classe malware, la precision indica la proporzione di file effettivamente malevoli tra quelli classificati come tali dal modello ed è calcolata come

$$\text{Precision}_{malware} = \frac{TP}{TP + FP}$$

La recall indica invece la proporzione di file malevoli correttamente classificati rispetto al totale dei file malevoli:

$$\text{Recall}_{malware} = \frac{TP}{TP + FN}$$

Per la classe goodware, la precision indica la proporzione di file effettivamente benigni tra quelli classificati come tali dal modello ed è calcolata come

$$\text{Precision}_{goodware} = \frac{TN}{TN + FN}$$

La recall indica invece la proporzione di file benigni correttamente classificati rispetto al totale dei file benigni:

$$\text{Recall}_{goodware} = \frac{TN}{TN + FP}$$

L'F1-score combina precision e recall attraverso la loro media armonica ed è calcolato, per entrambe le classi, come

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Infine, l'overall accuracy indica la proporzione di file classificati correttamente rispetto al numero totale di file analizzati ed è calcolata come

$$\text{Overall Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Una volta completati i test, i valori delle matrici di confusione e le metriche calcolate a partire da essi sono stati salvati in un file CSV insieme ai principali parametri delle configurazioni sperimentali utilizzate, in modo da distinguere le diverse configurazioni e i corrispondenti risultati, per poi confrontarne le prestazioni.

4.3 Risultati

Capitolo 5

Conclusioni e sviluppi futuri

Bibliografia

[1] Microsoft. PE format. `https://learn.microsoft.com/en-us/windows/win32/debug/pe-format`.

[2] Faiza Babar Khan, Muhammad Hanif Durad, Asifullah Khan, Farrukh Aslam Khan, Sajjad Hussain Chauhdary, and Mohammed Alqarni. Detection of data scarce malware using one-shot learning with relation network. *IEEE Access*, 11:74438–74457, 2023.

[3] Yunchun Zhang, Jiaqi Jiang, Chao Yi, Hai Li, Shaohui Min, Ruifeng Zuo, Zhenzhou An, and Yongtao Yu. A robust CNN for malware classification against executable adversarial attack. *Electronics*, 13(5):989, 2024.

[4] Jinrong Bai, Junfeng Wang, and Guozhong Zou. A malware detection scheme based on mining format information. *The Scientific World Journal*, 2014(1):260905, 2014.

[5] Murugiah Souppaya and Karen Scarfone. Guide to malware incident prevention and handling for desktops and laptops. Technical Report NIST SP 800-83 Rev. 1, National Institute of Standards and Technology, 2013.

[6] Manuel Egele, Theodoor Scholte, Engin Kirda, and Christopher Kruegel. A survey on automated dynamic malware-analysis techniques and tools. *ACM Computing Surveys*, 44(2):1–42, 2012.

[7] Daniele Ucci, Leonardo Aniello, and Roberto Baldoni. Survey of machine learning techniques for malware analysis. *Computers & Security*, 81:123–147, 2019.

[8] Lakshmanan Nataraj, Shanmugavadivel Karthikeyan, Grégoire Jacob, and B. S. Manjunath. Malware images: Visualization and automatic classification. *Proceedings of the 8th International Symposium on Visualization for Cyber Security*, 2011.

[9] Hui Guo, Shuguang Huang, Cheng Huang, Fan Shi, Min Zhang, and Zulie Pan. Binary file's visualization and entropy features analysis combined with multiple deep learning networks for malware classification. *Security and Communication Networks*, 2020:8881760, 2020.

[10] Vo Khuong Linh, Nguyen Viet Hung, Tran Ngoc Anh, Duong Do Nhuan, and Dinh Cong Hien. Enhance deep learning model for malware detection with a new image representation method. *Journal of Science and Technology on Information Security*, 1(21):31–39, 2024.

[11] Huy Nguyen, Fabio Di Troia, Genya Ishigaki, and Mark Stamp. Generative adversarial networks and image-based malware classification. *Journal of Computer Virology and Hacking Techniques*, 19:579–595, 2023.

[12] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015.

[13] Kh. Nafizul Haque. What is convolutional neural network — cnn (deep learning). Medium, February 2023.

[14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[15] Michael Lester. Pe malware machine learning dataset, 2021. Practical Security Analytics.

[16] PyTorch. Pytorch documentation. Documentazione ufficiale.

[17] Hugging Face. Resnet-50 v1.5. Model card del modello microsoft/resnet-50.

Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.